

DriveEase

Controllers Layer

MVC controllers: routing browser requests to logic and views
ASP.NET Core 9.0 MVC | Entity Framework Core | PostgreSQL (Supabase)

What the Controllers layer does

Controllers are the entry point of every browser request. In ASP.NET Core MVC a controller is a C# class (ending in **Controller**) whose public methods are **actions**. The routing system maps a URL to a controller + action, the action runs logic, and it returns an **ActionResult** (usually a View, a redirect, or JSON). DriveEase uses the default route {controller=Home}/{action=Index}/{id?} defined in Program.cs.

Files in this layer

File	Purpose	Status
HomeController.cs	Serves the landing page, privacy page and error page.	Present (default template)

HomeController

The only controller currently in the project. It inherits from Controller and receives an ILogger<HomeController> through constructor dependency injection (the framework supplies it automatically).

Constructor & dependencies

```
private readonly ILogger<HomeController> _logger;

public HomeController(ILogger<HomeController> logger)
{
    _logger = logger;
}
```

Actions

Action	HTTP / Route	Returns	What it does
Index()	GET / or /Home/Index	View()	Renders the home page (Views/Home/Index.cshtml).
Privacy()	GET /Home/Privacy	View()	Renders the privacy page.
Error()	GET /Home/Error	View(ErrorViewModel)	Builds an ErrorViewModel with the current request id and shows the shared error view. Decorated with [ResponseCache(NoStore)] so error pages are never cached.

The Error action in detail

```
[ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
public IActionResult Error()
{
}
```

```
return View(new ErrorViewModel {  
    RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });  
}
```

RequestId is taken from the current diagnostic Activity if present, otherwise from the HTTP trace identifier. This id is shown on the error page so a user can quote it in a bug report.

Current state & what is missing

The controller layer is still the stock ASP.NET template. The business services (User, Vehicle, Booking, Payment, Review, Admin) are fully written but **not yet reachable from the browser** because no controllers call them.

Controllers a complete build would add

Controller (suggested)	Would use service	Typical actions
AccountController	IUserService	Register (GET/POST), Login (GET/POST), Logout
VehiclesController	IVehicleService	Index/Search, Details, Recommend
BookingsController	IBookingService	Create, MyBookings, Cancel, price preview
PaymentController	IPaymentService	Pay (GET/POST), receipt
ReviewsController	IReviewService	AddReview (POST), vehicle reviews
AdminController	IAdminService, IVehicleService, IUserService	Dashboard, manage vehicles, manage users

How a controller would wire to a service

```
public class AccountController : Controller
{
    private readonly IUserService _users;
    public AccountController(IUserService users) => _users = users;

    [HttpPost]
    public async Task<IActionResult> Login(LoginViewModel vm)
    {
        if (!ModelState.IsValid) return View(vm);
        var (ok, msg, user) = await _users.LoginAsync(vm);
        if (!ok) { ModelState.AddModelError("", msg); return View(vm); }
        // set session / cookie, then redirect
        return RedirectToAction("Index", "Home");
    }
}
```

Note the services already return **(bool Success, string Message, ...)** tuples specifically so a controller can push the message into ModelState and redisplay the form. The plumbing on the service side is ready; only the controllers and views are missing.